TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: ALEXI RAFAEL YOSA CARRASCO

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: #1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad

Actividad # 1 Estudio de caso - Sistema de Control de Rescate y Adopción Animal

El estudiante deberá diseñar un sistema orientado a la gestión de casos en un refugio animal. El sistema debe permitir reportar ingresos de animales rescatados (perros, gatos, fauna silvestre), asignar voluntarios o veterinarios encargados y gestionar estados del proceso (ej. Rescatado, En Tratamiento, Adopción). Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

· Análisis del problema:

Actores Principales

- **Ciudadano / Reportante:** Usuario externo que reporta un animal en situación de calle, abandono o emergencia.
- **Voluntario / Veterinario:** Personal operativo encargado de atender las alertas, evaluar la salud del animal y actualizar su expediente médico.
- **Administrador del Refugio:** Gestiona los expedientes, aprueba solicitudes de adopción, asigna roles y supervisa las estadísticas del sistema.

Requerimientos Funcionales

1. **Reportar rescate:** Permitir registrar una alerta con ubicación, foto y condición del animal.
2. **Categorizar expediente:** Crear expedientes específicos según la especie (Perro, Gato, Fauna Silvestre).
3. **Asignar personal:** Asignar automáticamente o manualmente un veterinario o voluntario al caso.
4. **Gestión de estados:** Cambiar la condición del caso (Rescatado, En Tratamiento, Disponible para Adopción, Adoptado).
5. **Notificaciones:** Enviar alertas automáticas al personal cuando se genera un rescate cercano o prioritario.

Requerimientos No Funcionales

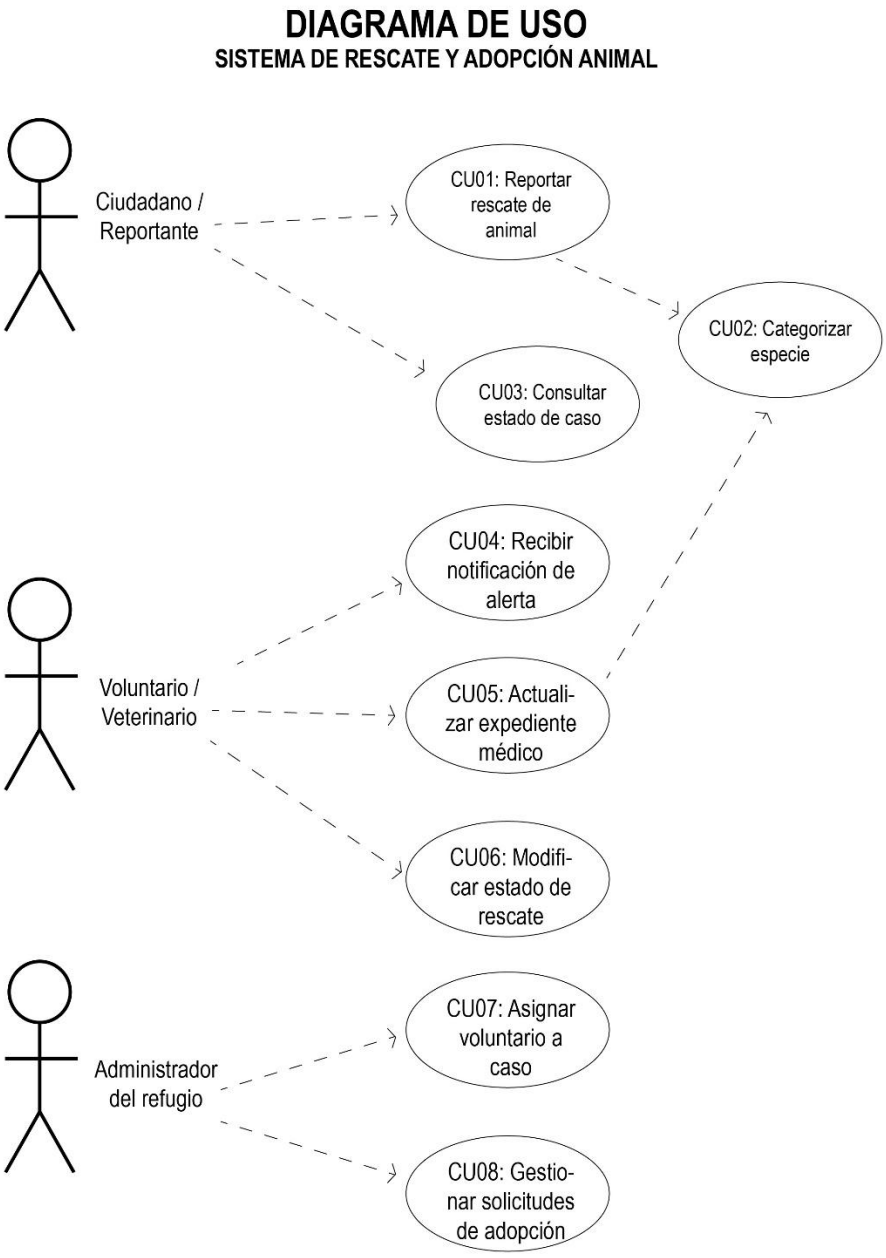
1. **Mantenibilidad y Escalabilidad:** Estructura modular orientada a objetos para agregar nuevas especies o roles sin romper el sistema.
2. **Trazabilidad de versiones:** Control del código fuente utilizando el flujo de trabajo profesional Gitflow.
3. **Disponibilidad y Consistencia:** Garantizar que la conexión a la base de datos sea única para evitar duplicación de información de los rescates.

· Diseño del sistema:

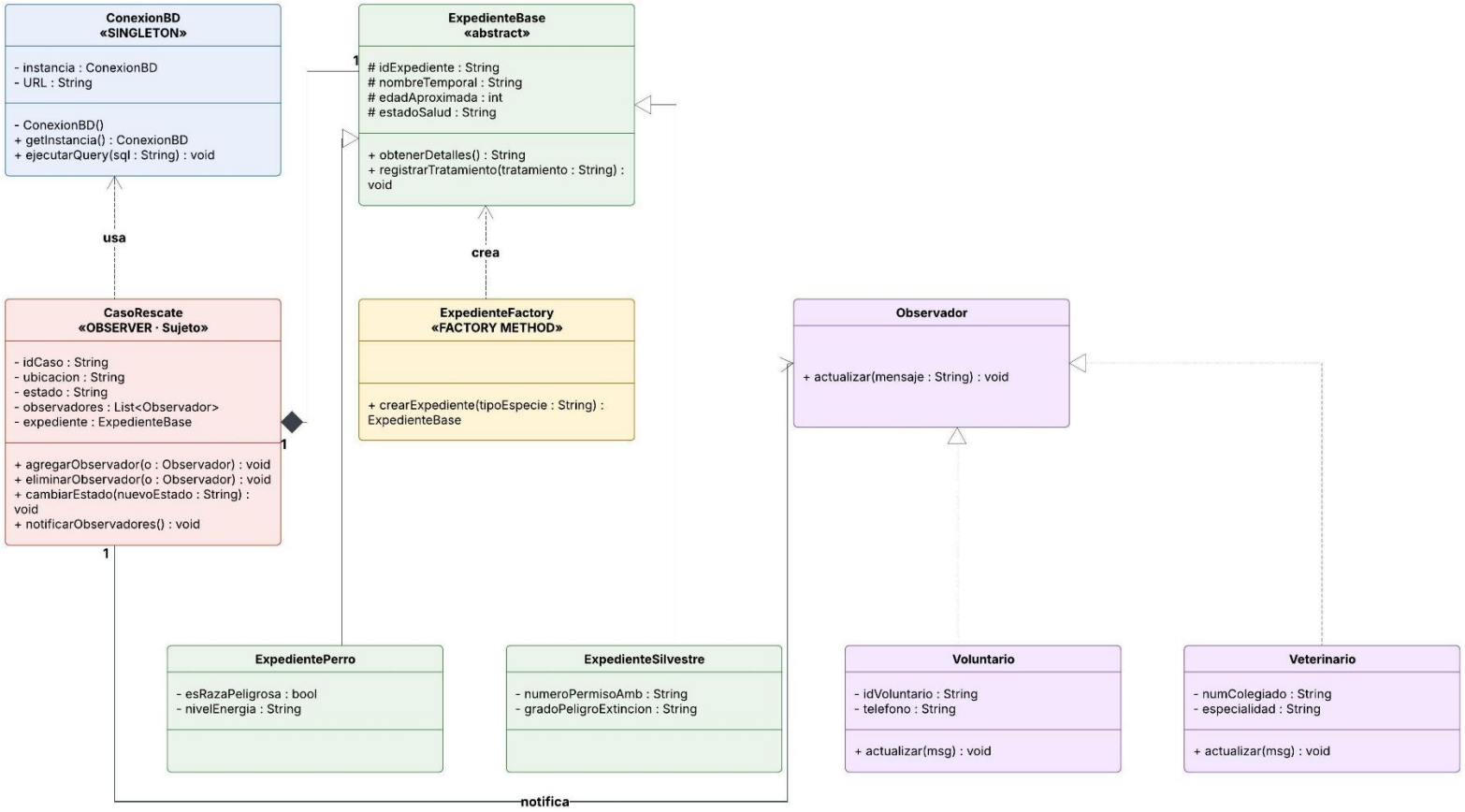
Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

https://github.com/alexiyosa-utm/AppPets

Diagrama de casos de uso



- **Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.**



Diseñar la arquitectura general del sistema integrando el flujo de versiones.

A. Capas de la Arquitectura del Sistema (MVC)

1. ***Capa de Presentación (Vista / Frontend):***
    - Componentes: Interfaz web/móvil donde el Ciudadano reporta rescates y los Voluntarios/Veterinarios gestionan casos.
    - Tecnología recomendada: HTML5/CSS3/JavaScript o React.
2. ***Capa de Control (Controlador / Backend API):***
    - Componentes: Recibe las peticiones HTTP de la vista, orquesta la lógica de negocio y responde con los datos procesados.
    - Tecnología recomendada: Node.js/Express, Python (FastAPI/Django) o Java (Spring Boot).
3. ***Capa de Lógica de Negocio y Patrones (Modelo):***
    - Componentes: Contiene las clases centrales (CasoRescate, ExpedienteFactory, ConexionBD). Aquí residen las reglas de negocio y los patrones de diseño (Singleton, Factory, Observer).
4. ***Capa de Datos (Base de Datos / Storage):***
    - Componentes: Almacena tablas de casos, usuarios, expedientes y registros de notificaciones.
    - Tecnología recomendada: PostgreSQL o MySQL.

B. Integración con el Flujo de Trabajo Gitflow

El desarrollo de cada capa y componente técnico de la arquitectura se gestiona mediante el modelo de ramas de Gitflow.

Flujo de trabajo por rama:

1. **main (Producción):** Mantiene únicamente la versión estable y funcional que el refugio usa en el entorno real.
2. **develop (Desarrollo):** Sirve como la rama integradora donde se unen todos los módulos probados.
3. **feature/ (Funcionalidades):** Cada desarrollador trabaja una tarea arquitectónica específica de forma aisada:
    - feature/modulo-conexion-bd $\rightarrow$ Implementación del patrón **Singleton**.
    - feature/expedientes-especies $\rightarrow$ Implementación del patrón **Factory Method**.
    - feature/sistema-notificaciones $\rightarrow$ Implementación del patrón **Observer**.

- feature/interfaz-usuario $\rightarrow$ Creación de las Vistas del sistema.

4. **Merge & Pull Requests:** Al finalizar una funcionalidad, se realiza un *Pull Request* hacia develop con revisión de código antes de autorizar el *merge*.

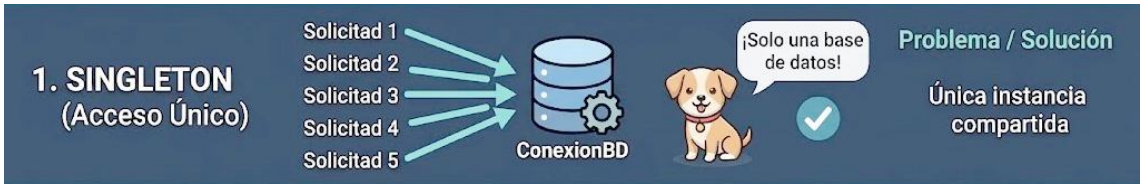
· Aplicación de patrones de diseño:

1. Patrón Singleton (Creacional)

Propósito y Justificación

En un sistema de gestión de refugios, abrir múltiples conexiones a la base de datos o instanciar repetidamente el motor central de casos consume recursos de memoria innecesarios y puede generar inconsistencias en los datos.

El patrón **Singleton** garantiza que exista **una única instancia** de la clase ConexionBD durante toda la ejecución de la aplicación, proporcionando un punto de acceso global a ella.



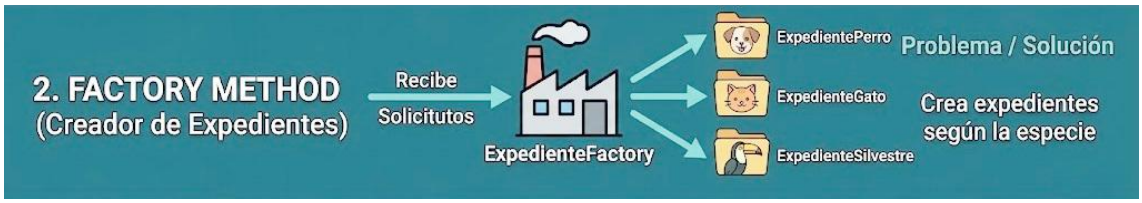
2. Patrón Factory Method (Creacional)

Propósito y Justificación

El refugio recibe distintas especies de animales (Perros, Gatos, Fauna Silvestre). Cada tipo de expediente requiere atributos y reglas de negocio particulares (por ejemplo, los perros requieren registro de raza/vacunas, mientras que la fauna silvestre requiere permisos del ministerio de ambiente).

Si creáramos estos objetos directamente con condicionales if/else en todo el código, el sistema sería rígido y difícil de extender. El patrón **Factory Method**

delega la responsabilidad de instanciar el tipo correcto de expediente a una clase fábrica (ExpedienteFactory), respetando el principio Open/Closed (OCP) de SOLID.

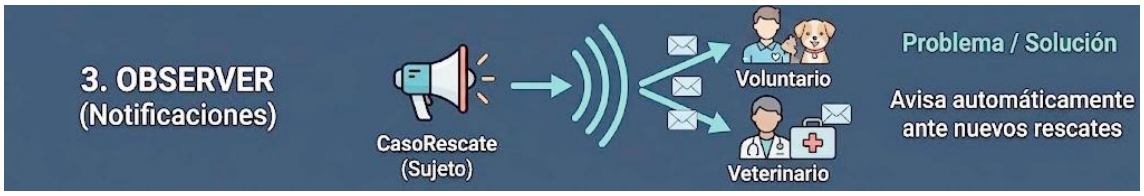


3. Patrón Observer (Comportamiento)

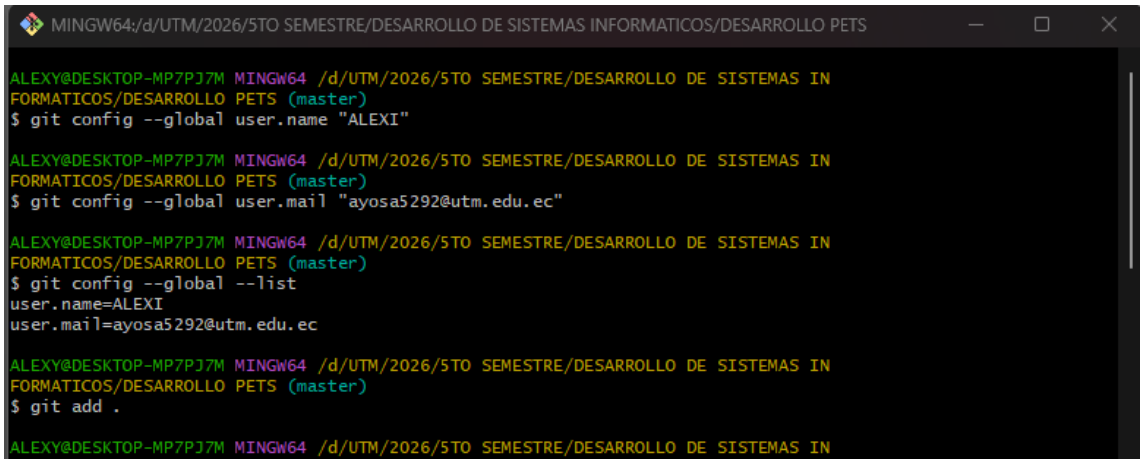
Propósito y Justificación

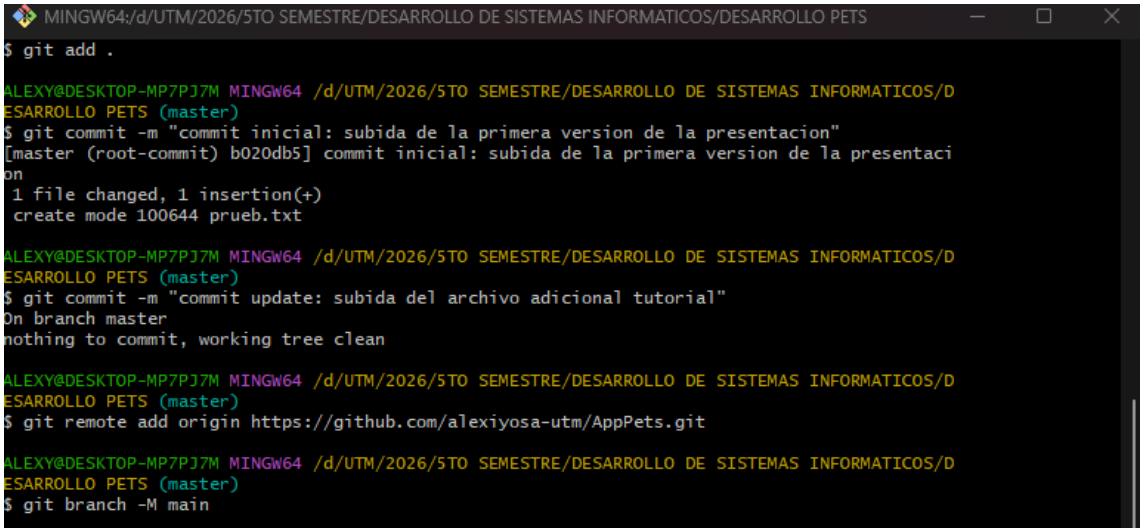
Cuando un ciudadano reporta un nuevo rescate en la plataforma, el sistema debe notificar de inmediato a los voluntarios y veterinarios disponibles para que acudan a la emergencia o preparen el área médica.

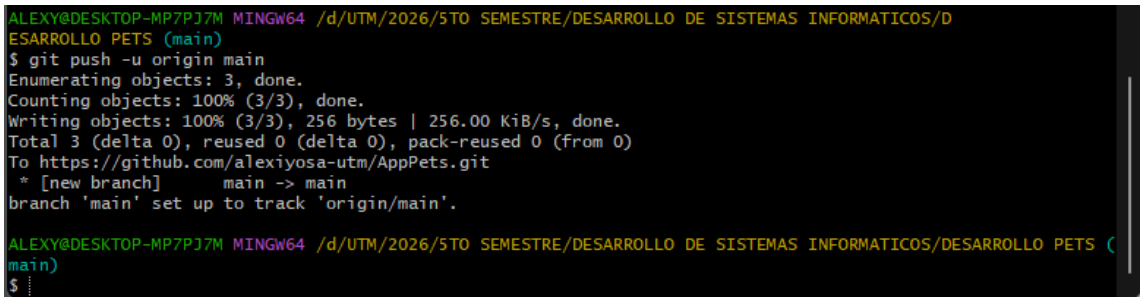
El patrón **Observer** establece una relación de uno a muchos entre el caso de rescate (Sujeto) y el personal del refugio (Observadores). Cuando el estado del rescate cambia o se genera un caso nuevo, todos los observadores registrados reciben una notificación automática, logrando un desacoplamiento total entre el módulo de rescates y el módulo de notificaciones.



Evidencias


MINGW64:/d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/DESARROLLO PETS
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS IN
FORMATICOS/DESARROLLO PETS (master)
$ git config --global user.name "ALEXI"
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS IN
FORMATICOS/DESARROLLO PETS (master)
$ git config --global user.mail "ayosa5292@utm.edu.ec"
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS IN
FORMATICOS/DESARROLLO PETS (master)
$ git config --global --list
user.name=ALEXI
user.mail=ayosa5292@utm.edu.ec
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS IN
FORMATICOS/DESARROLLO PETS (master)
$ git add .
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS IN


MINGW64:/d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/DESARROLLO PETS
$ git add .
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/D
ESARROLLO PETS (master)
$ git commit -m "commit inicial: subida de la primera version de la presentacion"
[master (root-commit) b020db5] commit inicial: subida de la primera version de la presentaci
on
1 file changed, 1 insertion(+)
create mode 100644 prueb.txt
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/D
ESARROLLO PETS (master)
$ git commit -m "commit update: subida del archivo adicional tutorial"
On branch master
nothing to commit, working tree clean
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/D
ESARROLLO PETS (master)
$ git remote add origin https://github.com/alexiyosa-utm/AppPets.git
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/D
ESARROLLO PETS (master)
$ git branch -M main


ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/D
ESARROLLO PETS (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 256 bytes | 256.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/alexiyosa-utm/AppPets.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
ALEXY@DESKTOP-MP7PJ7M MINGW64 /d/UTM/2026/5TO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/DESARROLLO PETS (
main)
$

Bibliografía

- Beynon-Davies, P. (2014). *Sistemas de información: Introducción a la informática en las organizaciones*. Editorial Reverté. https://api.pageplace.de/preview/DT0400.9788429194449_A39629297/preview-9788429194449_A39629297.pdf
- Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. https://dl.acm.org/doi/book/10.5555/186897
- Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media. https://dataintensive.net/
- Loeliger, J., & McCullough, M. (2012). *Version control with Git: Powerful tools and techniques for collaborative software development* (2.ª ed.). O'Reilly Media. https://www.goodreads.com/book/show/6548113-version-control-with-git
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall. https://www.oreilly.com/library/view/clean-code-a/9780136083238/
- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill Interamericana. https://books.google.com/books?id=N7YwAAAACAAJ
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill. https://github.com/FranklinAQP/SISE_2019_BD/blob/master/fundamentos%20de%20Bases-de-datos-silberschatz-korth-sudarshan.pdf
- Sommerville, I. (2011). *Software engineering* (9.ª ed.). Addison-Wesley. https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003310
- Tanenbaum, A. S., & Wetherall, D. J. (2012). *Redes de computadoras* (5.ª ed.). Pearson Educación. https://bibliotecadigital.utn.edu.ec/s/inicio/item/13705

https://lucid.app/lucidchart/7c7fd582-1315-4a16-b009-0cd07805a35b/edit?viewport_loc=-690%2C-351%2C3674%2C1940%2C0_0&invitationId=inv_36598e4f-847d-40ec-bca4-08bc8f4a6c41

https://github.com/alexiyosa-utm/AppPets